

TAREA — TRABAJO DE PLANEAMIENTO

Punto de Partida de tu Proyecto

Elige tu proyecto y describe cómo va hasta la Sesión 9

Desarrollo de Aplicaciones Básico

Alumno: [Tu nombre]

Proyecto: Zoapex

Fecha: 11 de junio de 2026

1. Elige tu proyecto

Mi proyecto se llama Zoapex y es una aplicación de escritorio de e-commerce para una tienda de mascotas. El prototipo se centra en mostrar el catálogo de productos y en registrar las ventas de esos productos, pero el diseño queda preparado para crecer hacia un sistema más completo. Elegí este rubro porque me permite cumplir de forma clara las tres condiciones que se aplicarán más adelante en el curso:

- **Mantenimientos CRUD:** gestiono el alta, edición y eliminación de productos y de categorías (y, en una fase posterior, de clientes).
- **Operación maestro-detalle transaccional:** el registro de un pedido, con una cabecera y varias líneas de detalle, que debe grabarse “todo o nada”. Mi pareja es Pedido → DetallePedido (en la base de datos, order → order_detail).
- **Consultas y reportes:** el catálogo ya combina las tablas producto y categoría, y a futuro tendré un reporte de ventas que cruza pedido, detalle, producto y cliente, pensado para poder exportarse a Excel.

2. Describe de qué trata tu proyecto

Zoapex es un sistema para una tienda de mascotas que ordena su catálogo y registra sus ventas. Lo usan dos perfiles: el administrador, que mantiene los productos y las categorías (precios, stock, descripciones, imagen), y el usuario final o vendedor, que navega el catálogo, arma un carrito y concreta la compra. Toda la información vive en una base de datos PostgreSQL alojada en Supabase (en la nube), y la aplicación de escritorio se conecta a ella para leer y grabar datos.

La operación más importante es el registro de la venta y ocurre así, paso a paso: el usuario abre el catálogo y ve los productos traídos desde Supabase; va agregando productos al carrito indicando la cantidad de cada uno; al confirmar, el sistema valida que haya stock suficiente, calcula el subtotal, le suma el IGV del 18 % y obtiene el total; finalmente graba el pedido junto con todas sus líneas de detalle de forma transaccional y descuenta el stock de cada producto. Si cualquier paso falla, no se guarda nada, manteniendo la base de datos consistente. Por eso mi pareja maestro-detalle es Pedido → DetallePedido (order → order_detail).

3. Cómo va mi proyecto hasta la Sesión 9

Apoyándome en lo visto en clase (Windows Forms y MDI, capas con CRUD en ADO.NET, reportes a Excel y la operación maestro-detalle transaccional), tengo definido el plano de mi sistema de la siguiente manera:

- **Arquitectura por capas:** el proyecto se organiza en 4 proyectos independientes. Zoapex.Entities contiene las clases POCO (es el núcleo y no referencia a nadie); Zoapex.DataAccess maneja la conexión a Supabase con el driver Npgsql y tiene una clase DAL por entidad; Zoapex.Business concentra todas las validaciones y el cálculo de totales; y Zoapex.Presentation es la interfaz en WPF. Las referencias van en una sola dirección: Presentation → Business → DataAccess → Entities, de modo que la interfaz nunca habla directo con la base de datos.
- **Tablas principales:** mis tablas maestras son category, product y customer. La pareja cabecera-detalle es order (cabecera del pedido) con order_detail (las líneas), enlazadas para guardar la venta completa.
- **Módulos previstos:** para el administrador, el CRUD de productos y de categorías y, más adelante, el reporte de ventas exportable a Excel. Para el usuario final, el catálogo visual, el carrito de compras y el registro de la venta con descuento automático de stock.
- **Estado actual:** lo más avanzado es el diseño de la arquitectura y la base de datos: ya tengo definidas las 5 tablas, 7 funciones de PostgreSQL (equivalentes a stored procedures) y los datos de prueba. Me falta construir el código de cada capa y las ventanas en WPF, y dejo para fases siguientes el login con roles, la gestión completa de clientes y los dashboards.

Como elementos diferenciadores, mi base de datos está en la nube (Supabase / PostgreSQL) en lugar de un SQL Server local, uso WPF en vez de Windows Forms y todo el código (clases, métodos y variables) está en inglés, dejando los comentarios en español.

4. ¿En qué tecnología hago el CRUD de administración?

Para mi proyecto elegí el esquema Form + Web. La parte administrativa (el CRUD de productos y categorías) la desarrollaré en escritorio con WPF, que en mi caso reemplaza a Windows Forms como diferenciador; y la cara pública que usará el cliente (el catálogo y la tienda) la llevaré a la web con ASP.NET Core en una fase posterior. Así, el administrador trabaja cómodo desde una app de escritorio y el cliente final puede comprar desde el navegador.

Respeto la regla de oro en todo momento: la interfaz nunca habla directo con la base de datos. Las capas Business, DataAccess y Entities se reutilizan exactamente igual en escritorio o en web, y lo único que cambia es la presentación. Por ahora esto es solo el plano: el CRUD web real lo construiré más adelante con Entity Framework y LINQ, tal como veremos en las Sesiones 12-13.

Parte 4 — Aplicación de la Sesión 12

Esta sección documenta la implementación de Entity Framework Core, LINQ, procedimientos almacenados (funciones PostgreSQL) y AJAX en la parte web del proyecto Zoapex, continuando el documento de planeamiento entregado en la PA3.

Decisión y justificación general

Para Zoapex mantuve la Ruta A (Escritorio + Web) definida en la PA3. El CRUD de productos, categorías y la operación maestro-detalle transaccional (Pedido → DetallePedido) se gestionan desde la aplicación de escritorio WPF con arquitectura en N capas y ADO.NET (Npgsql). La parte web (ASP.NET Core Razor Pages) se conecta a la misma base de datos en Supabase y permite al cliente ver el catálogo, iniciar sesión, armar un carrito y confirmar su compra.

En la web utilizo EF Core como ORM principal para mapear las tablas de PostgreSQL, LINQ para las consultas del catálogo e historial de pedidos del cliente, y la función `fn_register_order` como procedimiento almacenado para registrar la venta de forma transaccional (cabecera + detalle + descuento de stock). El AJAX lo apliqué en el catálogo y en el carrito para filtrar productos, ver el detalle y actualizar totales sin recargar la página.

Respeto la regla de oro: la vista nunca consulta la base de datos directamente. Las páginas Razor llaman a repositorios en la capa Data (CatalogRepository, OrderRepository, CustomerRepository), y estos usan EF Core, LINQ o la invocación al procedimiento almacenado.

A. Entity Framework Core — DbContext y modelo de datos

Justificación: uso EF Core para mapear las tablas de Supabase (category, product, customer, order, order_detail) como entidades C# y centralizar las relaciones entre ellas. Esto me permite reutilizar el mismo contexto en todos los repositorios de la capa web.

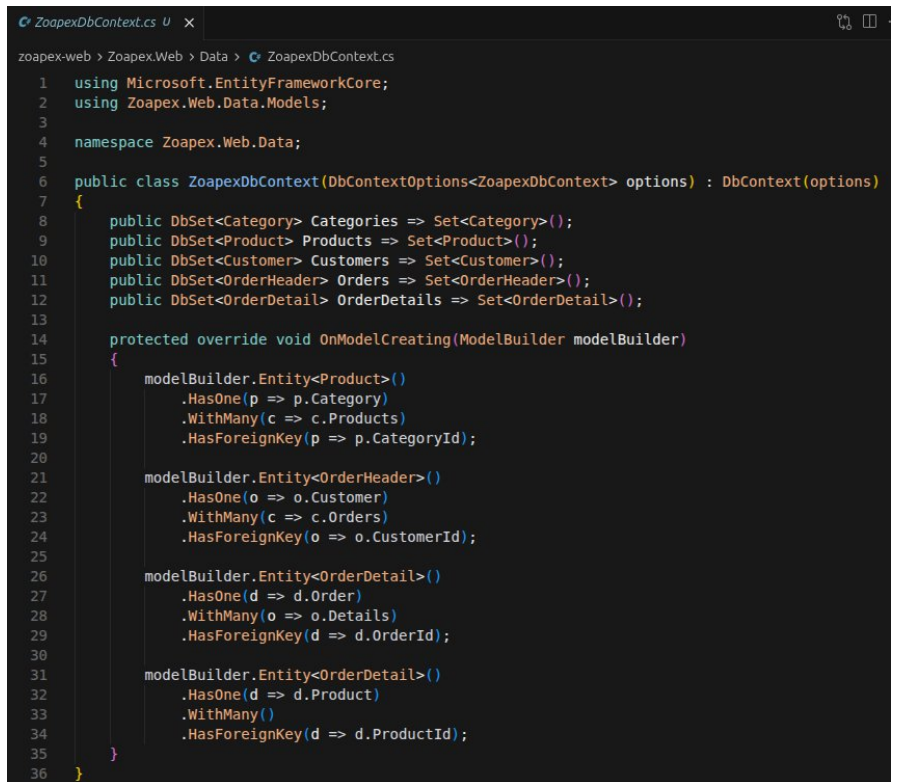
Archivo: Zoapex.Web/Data/ZoapexDbContext.cs

```
public class ZoapexDbContext(DbContextOptions<ZoapexDbContext> options) : DbContext(options)
{
    public DbSet<Category> Categories => Set<Category>();
    public DbSet<Product> Products => Set<Product>();
    public DbSet<Customer> Customers => Set<Customer>();
    public DbSet<OrderHeader> Orders => Set<OrderHeader>();
    public DbSet<OrderDetail> OrderDetails => Set<OrderDetail>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<Product>()
            .HasOne(p => p.Category)
            .WithMany(c => c.Products)
            .HasForeignKey(p => p.CategoryId);

        modelBuilder.Entity<OrderHeader>()
            .HasOne(o => o.Customer)
            .WithMany(c => c.Orders)
            .HasForeignKey(o => o.CustomerId);

        modelBuilder.Entity<OrderDetail>()
            .HasOne(d => d.Order)
            .WithMany(o => o.Details)
            .HasForeignKey(d => d.OrderId);
    }
}
```



```
1 using Microsoft.EntityFrameworkCore;
2 using Zoapex.Web.Data.Models;
3
4 namespace Zoapex.Web.Data;
5
6 public class ZoapexDbContext(DbContextOptions<ZoapexDbContext> options) : DbContext(options)
7 {
8     public DbSet<Category> Categories => Set<Category>();
9     public DbSet<Product> Products => Set<Product>();
10    public DbSet<Customer> Customers => Set<Customer>();
11    public DbSet<OrderHeader> Orders => Set<OrderHeader>();
12    public DbSet<OrderDetail> OrderDetails => Set<OrderDetail>();
13
14    protected override void OnModelCreating(ModelBuilder modelBuilder)
15    {
16        modelBuilder.Entity<Product>()
17            .HasOne(p => p.Category)
18            .WithMany(c => c.Products)
19            .HasForeignKey(p => p.CategoryId);
20
21        modelBuilder.Entity<OrderHeader>()
22            .HasOne(o => o.Customer)
23            .WithMany(c => c.Orders)
24            .HasForeignKey(o => o.CustomerId);
25
26        modelBuilder.Entity<OrderDetail>()
27            .HasOne(d => d.Order)
28            .WithMany(o => o.Details)
29            .HasForeignKey(d => d.OrderId);
30
31        modelBuilder.Entity<OrderDetail>()
32            .HasOne(d => d.Product)
33            .WithMany()
34            .HasForeignKey(d => d.ProductId);
35    }
36 }
```

Explicación: el DbContext define los DbSet de cada tabla y configura las relaciones Product → Category, Order → Customer y OrderDetail → Order/Product. Las entidades usan atributos [Table] y [Column] para mapear los nombres en snake_case de PostgreSQL.

B. LINQ — catálogo e historial de pedidos

Justificación: para listados y filtros frecuentes (catálogo por categoría, búsqueda por nombre/código y historial del cliente) uso LINQ to Entities porque la consulta queda legible en C# y el compilador valida los nombres de propiedades.

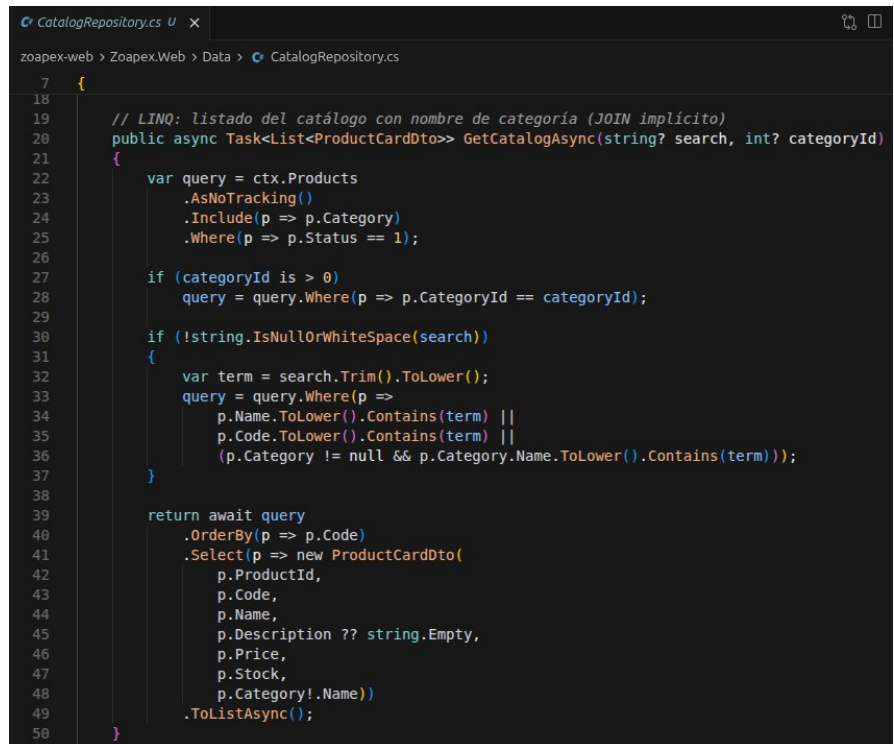
Archivo: Zoapex.Web/Data/CatalogRepository.cs — filtro del catálogo

```
public async Task<List<ProductCardDto>> GetCatalogAsync(string? search, int? categoryId)
{
    var query = ctx.Products
        .AsNoTracking()
        .Include(p => p.Category)
        .Where(p => p.Status == 1);

    if (categoryId is > 0)
        query = query.Where(p => p.CategoryId == categoryId);

    if (!string.IsNullOrEmpty(search))
    {
        var term = search.Trim().ToLower();
        query = query.Where(p =>
            p.Name.ToLower().Contains(term) ||
            p.Code.ToLower().Contains(term) ||
            (p.Category != null && p.Category.Name.ToLower().Contains(term)));
    }

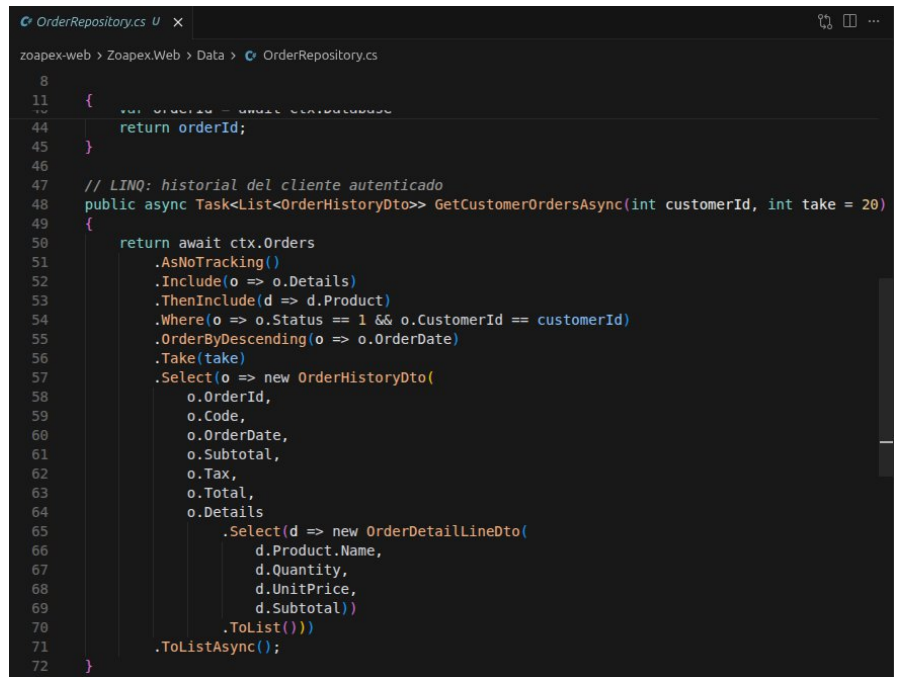
    return await query
        .OrderBy(p => p.Code)
        .Select(p => new ProductCardDto(...))
        .ToListAsync();
}
```



```
CatalogRepository.cs U X
zoapex-web > Zoapex.Web > Data > CatalogRepository.cs
7 {
18 // LINQ: listado del catálogo con nombre de categoría (JOIN implícito)
19 public async Task<List<ProductCardDto>> GetCatalogAsync(string? search, int? categoryId)
20 {
21     var query = ctx.Products
22         .AsNoTracking()
23         .Include(p => p.Category)
24         .Where(p => p.Status == 1);
25
26     if (categoryId is > 0)
27         query = query.Where(p => p.CategoryId == categoryId);
28
29     if (!string.IsNullOrEmpty(search))
30     {
31         var term = search.Trim().ToLower();
32         query = query.Where(p =>
33             p.Name.ToLower().Contains(term) ||
34             p.Code.ToLower().Contains(term) ||
35             (p.Category != null && p.Category.Name.ToLower().Contains(term)));
36     }
37
38     return await query
39         .OrderBy(p => p.Code)
40         .Select(p => new ProductCardDto(
41             p.ProductId,
42             p.Code,
43             p.Name,
44             p.Description ?? string.Empty,
45             p.Price,
46             p.Stock,
47             p.Category!.Name))
48         .ToListAsync();
49 }
50 }
```

Explicación: esta consulta LINQ filtra productos activos por categoría y texto de búsqueda, hace un JOIN implícito con Category mediante Include y proyecta el resultado a un DTO para la vista.

```
public async Task<List<OrderHistoryDto>> GetCustomerOrdersAsync(int customerId, int take = 20)
{
    return await ctx.Orders
        .AsNoTracking()
        .Include(o => o.Details)
        .ThenInclude(d => d.Product)
        .Where(o => o.Status == 1 && o.CustomerId == customerId)
        .OrderByDescending(o => o.OrderDate)
        .Take(take)
        .Select(o => new OrderHistoryDto(...))
        .ToListAsync();
}
```



The screenshot shows a code editor with a file named `OrderRepository.cs` open. The code is in C# and implements the `GetCustomerOrdersAsync` method. The method is asynchronous and returns a `Task<List<OrderHistoryDto>>`. It uses LINQ to query the `ctx.Orders` entity set. The query includes the `Details` property of the `Order` entity and the `Product` property of the `Detail` entity. It filters for orders with a status of 1 and a customer ID matching the provided `customerId`. The results are ordered by `OrderDate` in descending order and limited to `take` records. The results are then mapped to `OrderHistoryDto` objects, which include the order details and the product information. The method is implemented as follows:

```
8
11 {
12     // LINQ: historial del cliente autenticado
44     public async Task<List<OrderHistoryDto>> GetCustomerOrdersAsync(int customerId, int take = 20)
45     {
46         return await ctx.Orders
47             .AsNoTracking()
48             .Include(o => o.Details)
49             .ThenInclude(d => d.Product)
50             .Where(o => o.Status == 1 && o.CustomerId == customerId)
51             .OrderByDescending(o => o.OrderDate)
52             .Take(take)
53             .Select(o => new OrderHistoryDto(
54                 o.OrderId,
55                 o.Code,
56                 o.OrderDate,
57                 o.Subtotal,
58                 o.Tax,
59                 o.Total,
60                 o.Details
61                 .Select(d => new OrderDetailLineDto(
62                     d.Product.Name,
63                     d.Quantity,
64                     d.UnitPrice,
65                     d.Subtotal))
66                 .ToList())
67             .ToListAsync();
68     }
69 }
```

Explicación: trae los pedidos del cliente autenticado con sus líneas de detalle y el nombre del producto, ordenados del más reciente al más antiguo. Se usa en la página Mis pedidos.

C. Procedimiento almacenado — registro transaccional del pedido

Justificación: el registro de la venta (cabecera + detalle + cálculo de IGV + descuento de stock) involucra varias operaciones que deben ejecutarse 'todo o nada'. Por eso delego esa lógica a la función `fn_register_order` en PostgreSQL (equivalente a un stored procedure en SQL Server), reutilizando la misma función que ya usa la aplicación de escritorio WPF.

1) Función creada en Supabase (`zoapex-db/sql/zoapex_database.sql`)

```
CREATE OR REPLACE FUNCTION fn_register_order(
  p_customer_id INT,
  p_details JSONB
)
RETURNS INT AS $$
BEGIN
  -- Calcula subtotal, IGV (18%) y total
  -- Inserta cabecera en "order"
  -- Inserta líneas en order_detail
  -- Descuenta stock de cada producto
  RETURN v_order_id;
END;
$$ LANGUAGE plpgsql;
```



```
zoapex_database.sql M x  Cursor Settings  VS Code Settings

zoapex-db > sql > zoapex_database.sql

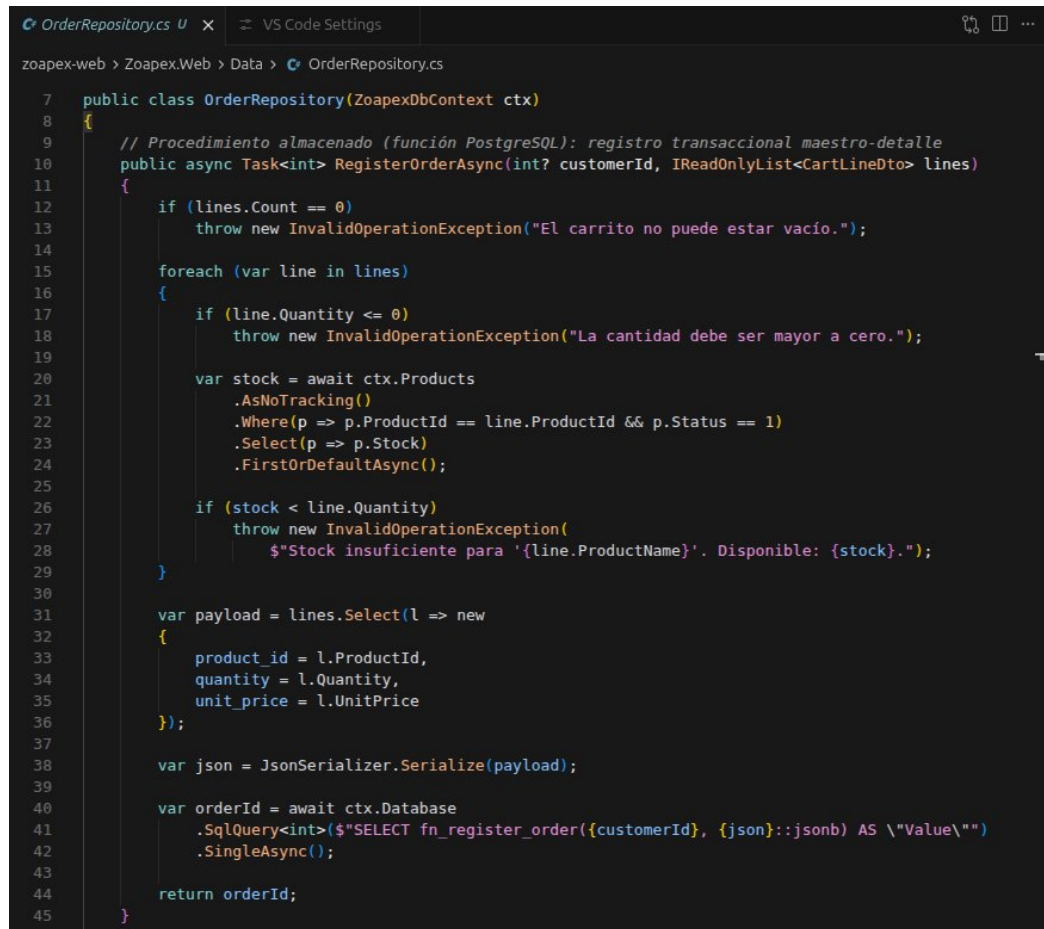
197 -- fn_register_order: Registra pedido + detalle en transacción y descuenta stock
198 -- p_customer_id puede ser NULL para ventas sin cliente registrado (mostrador)
199 CREATE OR REPLACE FUNCTION fn_register_order(
200   p_customer_id INT,
201   p_details JSONB -- [{product_id, quantity, unit_price}]
202 )
203 RETURNS INT AS $$
204 DECLARE
205   v_order_id INT;
206   v_next_id INT;
207   v_code VARCHAR(12);
208   v_subtotal NUMERIC(10,2) := 0;
209   v_tax NUMERIC(10,2);
210   v_total NUMERIC(10,2);
211   v_detail JSONB;
212   v_qty INT;
213   v_price NUMERIC(10,2);
214   v_prod_id INT;
215   v_line_subtotal NUMERIC(10,2);
216 BEGIN
217   -- Generar código del pedido
218   SELECT COALESCE(MAX(order_id), 0) + 1 INTO v_next_id FROM "order";
219   v_code := 'ORD' || LPAD(v_next_id::TEXT, 3, '0');
220
221   -- Calcular subtotal recorriendo el JSON
222   FOR v_detail IN SELECT * FROM jsonb_array_elements(p_details)
223   LOOP
224     v_qty := (v_detail->'quantity')::INT;
225     v_price := (v_detail->'unit_price')::NUMERIC(10,2);
226     v_subtotal := v_subtotal + (v_qty * v_price);
227   END LOOP;
228
229   v_tax := ROUND(v_subtotal * 0.18, 2);
230   v_total := v_subtotal + v_tax;
231
232   -- Insertar la orden
233   INSERT INTO "order" (code, customer_id, subtotal, tax, total, status)
234   VALUES (v_code, p_customer_id, v_subtotal, v_tax, v_total, 1)
235   RETURNING order_id INTO v_order_id;
236
237   -- Insertar detalles y descontar stock
238   FOR v_detail IN SELECT * FROM jsonb_array_elements(p_details)
239   LOOP
240     v_prod_id := (v_detail->'product_id')::INT;
241     v_qty := (v_detail->'quantity')::INT;
242     v_price := (v_detail->'unit_price')::NUMERIC(10,2);
243     v_line_subtotal := v_qty * v_price;
244
245     INSERT INTO order_detail (order_id, product_id, quantity, unit_price, subtotal)
246     VALUES (v_order_id, v_prod_id, v_qty, v_price, v_line_subtotal);
247
248     UPDATE product SET stock = stock - v_qty WHERE product_id = v_prod_id;
249   END LOOP;
250
251   RETURN v_order_id;
252 END;
253 $$ LANGUAGE plpgsql;
```


2) Invocación desde el proyecto web (EF Core + SqlQuery)

```
var json = JsonSerializer.Serialize(payload);
```

```
var orderId = await ctx.Database  
    .SqlQuery<int>($"SELECT fn_register_order({customerId}, {json}::jsonb) AS \"Value\"")  
    .SingleAsync();
```

```
return orderId;
```



```
7 public class OrderRepository(ZoapexDbContext ctx)
8 {
9     // Procedimiento almacenado (función PostgreSQL): registro transaccional maestro-detalle
10    public async Task<int> RegisterOrderAsync(int? customerId, IReadOnlyList<CartLineDto> lines)
11    {
12        if (lines.Count == 0)
13            throw new InvalidOperationException("El carrito no puede estar vacío.");
14
15        foreach (var line in lines)
16        {
17            if (line.Quantity <= 0)
18                throw new InvalidOperationException("La cantidad debe ser mayor a cero.");
19
20            var stock = await ctx.Products
21                .AsNoTracking()
22                .Where(p => p.ProductId == line.ProductId && p.Status == 1)
23                .Select(p => p.Stock)
24                .FirstOrDefaultAsync();
25
26            if (stock < line.Quantity)
27                throw new InvalidOperationException(
28                    $"Stock insuficiente para '{line.ProductName}'. Disponible: {stock}.");
29        }
30
31        var payload = lines.Select(l => new
32        {
33            product_id = l.ProductId,
34            quantity = l.Quantity,
35            unit_price = l.UnitPrice
36        });
37
38        var json = JsonSerializer.Serialize(payload);
39
40        var orderId = await ctx.Database
41            .SqlQuery<int>($"SELECT fn_register_order({customerId}, {json}::jsonb) AS \"Value\"")
42            .SingleAsync();
43
44        return orderId;
45    }
46 }
```

Explicación: la capa web serializa las líneas del carrito a JSON, invoca `fn_register_order` mediante `SqlQuery` y recibe el ID del pedido registrado. Si algo falla en la base de datos, no se confirma la venta.

D. AJAX (obligatorio) — catálogo y carrito sin recargar

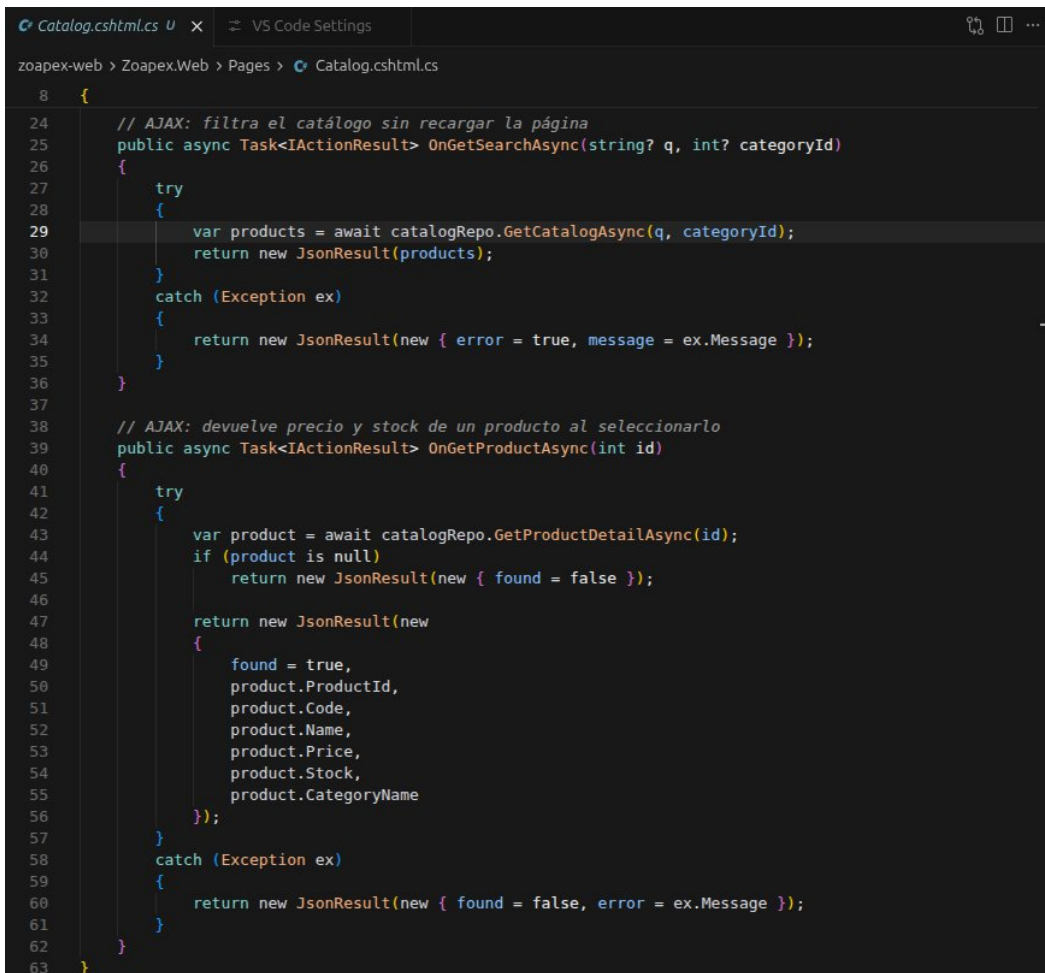
Justificación: al navegar el catálogo el cliente necesita filtrar productos y ver precio/stock al instante. En el carrito, al quitar un ítem debe actualizarse el resumen de totales sin recargar toda la página. Por eso implementé handlers Razor Pages que devuelven JSON y fetch en JavaScript.

1) Handler AJAX en el code-behind (Catalog.cshtml.cs)

```
public async Task<IActionResult> OnGetSearchAsync(string? q, int? categoryId)
{
    var products = await catalogRepo.GetCatalogAsync(q, categoryId);
    return new JsonResult(products);
}

public async Task<IActionResult> OnGetProductAsync(int id)
{
    var product = await catalogRepo.GetProductDetailAsync(id);
    if (product is null)
        return new JsonResult(new { found = false });

    return new JsonResult(new { found = true, product.Name, product.Price, product.Stock, ... });
}
```



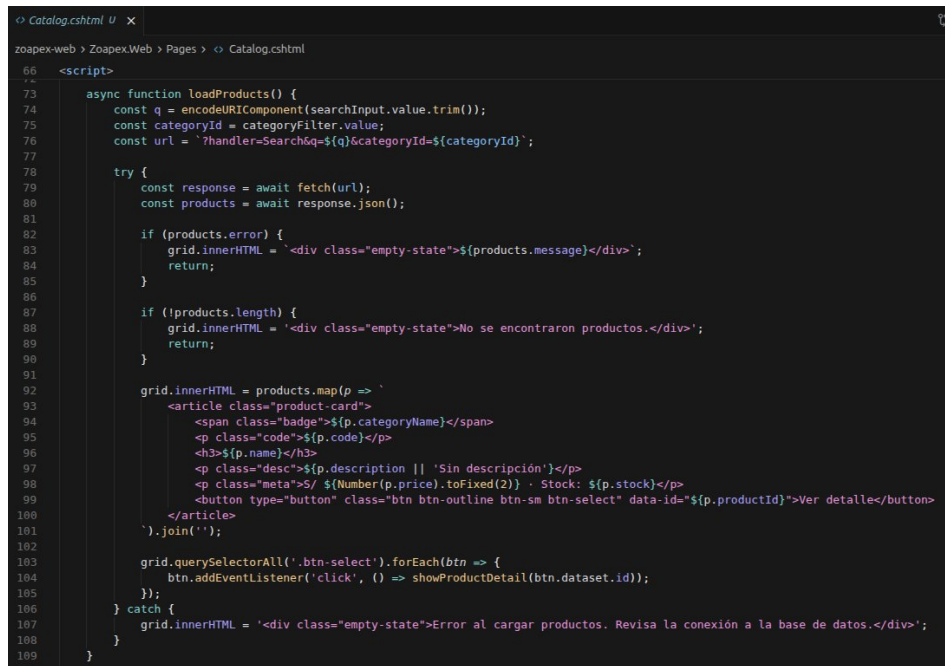
The screenshot shows a Visual Studio Code editor with a file named `Catalog.cshtml.cs` open. The code is written in C# and implements two asynchronous handlers for AJAX requests. The first handler, `OnGetSearchAsync`, filters the catalog by a search query and category ID. The second handler, `OnGetProductAsync`, retrieves the details of a specific product by its ID. Both handlers use `await` to call repository methods and return the results as `JsonResult` objects. The code is formatted with syntax highlighting and line numbers.

```
8  {
24 // AJAX: filtra el catálogo sin recargar la página
25 public async Task<IActionResult> OnGetSearchAsync(string? q, int? categoryId)
26 {
27     try
28     {
29         var products = await catalogRepo.GetCatalogAsync(q, categoryId);
30         return new JsonResult(products);
31     }
32     catch (Exception ex)
33     {
34         return new JsonResult(new { error = true, message = ex.Message });
35     }
36 }
37
38 // AJAX: devuelve precio y stock de un producto al seleccionarlo
39 public async Task<IActionResult> OnGetProductAsync(int id)
40 {
41     try
42     {
43         var product = await catalogRepo.GetProductDetailAsync(id);
44         if (product is null)
45             return new JsonResult(new { found = false });
46
47         return new JsonResult(new
48         {
49             found = true,
50             product.ProductId,
51             product.Code,
52             product.Name,
53             product.Price,
54             product.Stock,
55             product.CategoryName
56         });
57     }
58     catch (Exception ex)
59     {
60         return new JsonResult(new { found = false, error = ex.Message });
61     }
62 }
63 }
```

2) Fetch en la vista (Catalog.cshtml)

```
async function loadProducts() {  
  const url = `?handler=Search&q=${q}&categoryId=${categoryId}`;  
  const response = await fetch(url);  
  const products = await response.json();  
  // actualiza solo la grilla de productos, sin recargar la página  
  grid.innerHTML = products.map(p => `...`).join("");  
}
```

```
async function showProductDetail(id) {  
  const response = await fetch(`?handler=Product&id=${id}`);  
  const data = await response.json();  
  // actualiza nombre, precio y stock en el panel lateral  
}
```



```
<script>  
66  
67  
68  
69  
70  
71  
72  
73 async function loadProducts() {  
74   const q = encodeURIComponent(searchInput.value.trim());  
75   const categoryId = categoryFilter.value;  
76   const url = `?handler=Search&q=${q}&categoryId=${categoryId}`;  
77  
78   try {  
79     const response = await fetch(url);  
80     const products = await response.json();  
81  
82     if (products.error) {  
83       grid.innerHTML = `<div class="empty-state">${products.message}</div>`;  
84       return;  
85     }  
86  
87     if (!products.length) {  
88       grid.innerHTML = `<div class="empty-state">No se encontraron productos.</div>`;  
89       return;  
90     }  
91  
92     grid.innerHTML = products.map(p => `  
93       <article class="product-card">  
94         <span class="badge">${p.categoryName}</span>  
95         <p class="code">${p.code}</p>  
96         <h3>${p.name}</h3>  
97         <p class="desc">${p.description || 'Sin descripción'}</p>  
98         <p class="meta">$ / $(Number(p.price).toFixed(2)) · Stock: ${p.stock}</p>  
99         <button type="button" class="btn btn-outline btn-sm btn-select" data-id="${p.productId}">Ver detalles</button>  
100       </article>  
101     `).join('');  
102  
103     grid.querySelectorAll('.btn-select').forEach(btn => {  
104       btn.addEventListener('click', () => showProductDetail(btn.dataset.id));  
105     });  
106   } catch {  
107     grid.innerHTML = `<div class="empty-state">Error al cargar productos. Revisa la conexión a la base de datos.</div>`;  
108   }  
109 }  
110
```

Explicación: al escribir en el buscador o cambiar la categoría, fetch llama al handler OnGetSearchAsync y actualiza solo la grilla de productos. Al pulsar 'Ver detalle', OnGetProductAsync devuelve el precio y stock sin recargar la página. En el carrito, refreshCart() usa ?handler=Summary para recalcular subtotal, IGV y total vía AJAX.



[Inicio](#)
[Catálogo](#)
[Carrito](#)

[Iniciar sesión](#)
[Registrarse](#)



Bienvenido de vuelta

Inicia sesión para comprar y ver tus pedidos.

Correo electrónico

Contraseña

[Iniciar sesión](#)

¿No tienes cuenta? [Regístrate aquí](#)

Cuenta demo

Correo: cliente@zoapex.com

Contraseña: zoapex123

E. Login de cliente (fase web)

Complemento planificado en PA3: la web incluye registro e inicio de sesión de clientes. CustomerRepository valida credenciales con LINQ, PasswordHelper encripta la contraseña y CustomerSession guarda la sesión. El checkout del carrito requiere login y asocia el pedido al customer_id del cliente autenticado.

```

<> Login.cshtml U x
zoapex-web > Zoapex.Web > Pages > Account > <> Login.cshtml
1  @page
2  @model Zoapex.Web.Pages.Account.LoginModel
3  @{
4      ViewData["Title"] = "Iniciar sesión";
5  }
6
7  <section class="auth-card">
8      <div class="auth-header">
9          <span class="auth-icon">🐾</span>
10         <h1>Bienvenido de vuelta</h1>
11         <p>Inicia sesión para comprar y ver tus pedidos.</p>
12     </div>
13
14     @if (!string.IsNullOrEmpty(Model.ErrorMessage))
15     {
16         <div class="alert alert-error">@Model.ErrorMessage</div>
17     }
18
19     <form method="post" class="auth-form">
20         <input type="hidden" name="returnUrl" value="@Model.ReturnUrl" />
21
22         <label>
23             Correo electrónico
24             <input asp-for="Email" type="email" placeholder="tu@correo.com" required />
25         </label>
26
27         <label>
28             Contraseña
29             <input asp-for="Password" type="password" placeholder="*****" required />
30         </label>
31
32         <button type="submit" class="btn btn-primary btn-block">Iniciar sesión</button>
33     </form>
34
35     <p class="auth-footer">
36         ¿No tienes cuenta? <a asp-page="/Account/Register">Regístrate aquí</a>
37     </p>
38
39     <div class="demo-box">
40         <strong>Cuenta demo</strong>
41         <p>Correo: <code>cliente@zoapex.com</code></p>
42         <p>Contraseña: <code>zoapex123</code></p>
43     </div>
44 </section>

```